HPE Product Catalog – Deployment Guide

Step-by-step instructions to take the React + FastAPI + MySQL + Redis catalog from source code to running on Kubernetes (EKS) with GitLab CI/CD, ArgoCD GitOps, and Prometheus/Grafana/EFK monitoring. Based on the repository at `microservices/`.

0 What you are deploying

frontend
React 18 + Vite, served by nginx:1.27 on port 80. Proxies `/api/*` to the API (prefix stripped).

api
FastAPI on port 8000 (Python 3.11). Read-through Redis cache, MySQL via SQLAlchemy. `/health` checks both.

mysql
MySQL 8.4 with schema and 10 seed products in `mysql/init/01-schema.sql`. StatefulSet, 5Gi gp3 volume.

redis
Redis 7.4, password via `--requirepass`, 128mb LRU cache.

All four images are published to Docker Hub as one repository with prefixed tags: `kharatramesh/hpe:{api,mysql,redis,frontend}-2.0.<pipeline>`.

Two ways to run it

Path	Use for	Section
Docker Compose	Local laptop or a single server	Step 1
EKS + GitLab CI + ArgoCD	dev, uat and prod environments (the main pipeline)	Steps 2–10

Delivery flow (branch `master`)

ruff | hadolint | pytest → SAST + secrets → build → package x4 → Trivy x4 → publish to Docker Hub → deploy dev → smoke test → deploy uat → UAT tests → ZAP scan → deploy prod

> **Note**

> The repository `README.md` still describes an older EC2 / `docker compose` pipeline and `deploy/remote-deploy.sh`. The current `.gitlab-ci.yml` deploys through ArgoCD to Kubernetes, and this guide follows that. The EC2 script is unused by the pipeline.

★ Prerequisites

Need	Details
Tools on your workstation	`git`, `kubectl`, `helm`, `openssl`, `argocd` CLI (v2.13.x), AWS CLI + `eksctl`; Docker for the local step
Kubernetes cluster	EKS cluster `hpcluster`. Nodes need room: monitoring plus three environments is heavy (t3.medium allows only 17 pods per node, per `install.sh`).
Cluster add-ons	ingress-nginx controller, AWS EBS CSI driver, and a StorageClass named **gp3** (referenced by MySQL, Prometheus, Grafana, Elasticsearch)
DNS	Control of `priartw.com` (or your own domain, see Step 11): CNAMEs for `dev`, `uat`, `prod`, `argocd`, `grafana`, `kibana`
Accounts	GitLab project (`gitlab.com/kharatramesh/hpeproject`), Docker Hub account with write access to `kharatramesh/hpe`

1 Run locally with Docker Compose (optional smoke check)

Quickest way to confirm the images and app behave before touching the cluster.

Copy

```
cd microservices
cp .env.example .env
# edit .env: replace every change-me-* password
docker compose up --build
```

- App: `http://localhost/` API docs: `http://localhost:8000/docs`
- MySQL and Redis publish no host ports.

Copy

```
curl -s localhost/api/health
curl -s localhost/api/products
```

```
curl -s localhost:8000/products/1     # "source":"db", then "cache" on the second call
docker compose down                   # never add -v unless you want to delete the data
```

> **Gotcha**
> The MySQL init script runs only when the data volume (`mysql-data-v2`) is empty. After changing the schema or DB credentials, use a new volume name or delete the volume on purpose.

Run the API tests (no Docker needed)

Copy

```
python -m venv ~/venvs/microservices-api
~/venvs/microservices-api/bin/pip install -r api/requirements-dev.txt   # Windows: Scripts\p
cd api && ~/venvs/microservices-api/bin/python -m pytest -v
cd .. && ruff check --config api/ruff.toml api && ruff format --check --config api/ruff.toml
```

2 Prepare the Kubernetes cluster

Skip anything your cluster already has.

2.1 Connect kubectl

Copy

```
aws eks update-kubeconfig --name hpcluster --region <your-region>
kubectl get nodes
```

2.2 EBS CSI driver and gp3 StorageClass

Copy

```
kubectl get storageclass            # is there one named "gp3"?

cat <<'EOF' | kubectl apply -f -
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata: {name: gp3}
provisioner: ebs.csi.aws.com
parameters: {type: gp3}
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
EOF
```

> **Why it matters**

> MySQL, Prometheus, Grafana and Elasticsearch all request `storageClassName: gp3`. If it is missing, their PVCs stay `Pending` forever. The EBS CSI driver add-on (with an IAM role) must be installed for volumes to provision.

2.3 Ingress controller

```
helm upgrade --install ingress-nginx ingress-nginx \
  --repo https://kubernetes.github.io/ingress-nginx \
  --namespace ingress-nginx --create-namespace

kubectl -n ingress-nginx get svc ingress-nginx-controller   # note EXTERNAL-IP / hostname
```

Copy

2.4 Create the environment namespaces

ArgoCD can create them (`CreateNamespace=true`), but the secrets in Step 3 need them first.

```
kubectl create namespace dev
kubectl create namespace uat
kubectl create namespace prod
```

Copy

3 Create application secrets

Passwords are never stored in git. One script creates `app-secrets` in dev, uat and prod with random values.

```
bash k8s/create-secrets.sh
kubectl -n dev get secret app-secrets
```

Copy

Each secret holds `MYSQL_PASSWORD`, `MYSQL_ROOT_PASSWORD` and `REDIS_PASSWORD`. The script skips namespaces that already have one.

> **Gotcha**
> Changing MySQL passwords after the first start does *not* change the database. To reset, delete the StatefulSet's PVC (data loss) and recreate.

Non-secret settings (hosts, ports, cache TTL, log level) live in `k8s/base/config.yaml` as ConfigMap `app-config`. `AUTO_CREATE_TABLES` must stay `"false"`.

4 Install ArgoCD

```
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/v2.13.3/manifes
kubectl -n argocd rollout status deploy/argocd-server
```

Copy

4.1 Expose it through the ingress (HTTP)

The pipeline calls the CLI with `--plaintext --grpc-web`, so the server runs in insecure mode behind nginx.

```
kubectl apply -f argocd/argocd-ingress.yaml        # sets server.insecure=true and ho
kubectl -n argocd rollout restart deploy/argocd-server
```

Copy

4.2 First login and CI token

```
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath='{.data.password}' | as

argocd login argocd.priartw.com --plaintext --grpc-web --username admin
# create a dedicated CI account (recommended): add to argocd-cm  data: accounts.ci: apiKey
argocd account generate-token --account ci --plaintext --grpc-web
```

Copy

> **Needed later**
> Keep the token and the hostname: they become the GitLab variables `ARGOCD_AUTH_TOKEN` and `ARGOCD_SERVER`. The CI account needs permission to `get/sync/override` the `hpe-*` applications (RBAC in `argocd-rbac-cm`).

4.3 Let ArgoCD read the repository

If the GitLab project is private, add a repo credential (a read-only deploy token):

```
argocd repo add https://gitlab.com/kharatramesh/hpeproject.git \
  --username <deploy-token-user> --password <deploy-token> --plaintext --grpc-web
```

Copy

5 Register the three ArgoCD applications

One Application per environment, each pointing at a Kustomize overlay on `master`.

```
kubectl apply -f argocd/app-dev.yaml -f argocd/app-uat.yaml -f argocd/app-prod.yaml
kubectl -n argocd get applications
```

Copy

App	Overlay path	Namespace	Host	Replicas
hpe-dev	`k8s/overlays/dev`	dev	dev.priartw.com	api 1, frontend 1
hpe-uat	`k8s/overlays/uat`	uat	uat.priartw.com	api 1, frontend 1
hpe-prod	`k8s/overlays/prod`	prod	prod.priartw.com	api 2, frontend 2

> **How sync works**
> The Applications have no automated sync policy. The pipeline overrides the four image tags with `argocd app set --kustomize-image ...`, then runs `argocd app sync` and `argocd app wait --health`. Until the first pipeline run they show *OutOfSync*, and images fall back to the tags in `k8s/base/kustomization.yaml` (`*-2.0`), which only exist if you pushed them manually.

6 Configure GitLab

6.1 Push the code

```
cd microservices
git remote -v      # origin = https://gitlab.com/kharatramesh/hpeproject.git
git push -u origin master
```

Copy

6.2 CI/CD variables

Settings → CI/CD → Variables. Mark secrets *Masked*.

Variable	Value	Masked
`DOCKERHUB_USER`	Docker Hub user that owns `kharatramesh/hpe`	no
`DOCKERHUB_TOKEN`	Docker Hub access token with write scope	yes
`ARGOCD_SERVER`	`argocd.priartw.com` (no scheme)	no
`ARGOCD_AUTH_TOKEN`	token from Step 4.2	yes

`CI_REGISTRY*` variables are provided by GitLab.

> **Protected variables**
> If you tick *Protected*, `master` must be a protected branch, otherwise the variables arrive empty. The publish job fails fast with that exact hint.

6.3 Runner

A runner with privileged Docker-in-Docker (GitLab.com shared runners work once the account is verified). Package and publish jobs use `docker:27.3.1-dind`.

6.4 Docker Hub repository

Create `kharatramesh/hpe` (public is simplest: the cluster pulls anonymously). If private, add an `imagePullSecret` to each namespace's default ServiceAccount.

7 DNS

Create CNAME records pointing at the ingress load balancer hostname from Step 2.3:

Record	Target
dev.priartw.com, uat.priartw.com, prod.priartw.com	EXTERNAL-IP / hostname of `ingress-nginx-controller`
argocd.priartw.com	
grafana.priartw.com, kibana.priartw.com	

```
nslookup dev.priartw.com
```

Copy

8 Run the pipeline

Any push to `master` starts it (merge requests run the first stages only).

Stage	Jobs	Gate
quality	ruff, hadolint, pytest	any lint error / failing test

test	semgrep-sast, secret_detection	template rules
build	build-frontend (`npm ci && npm run build`), build-api (import check)	build error
package	x4: builds each image, pushes `:<commit sha>` to GitLab registry	build / push error
scan	Trivy x4	fixable CRITICAL CVE (HIGH only reported)
publish	x4: tags `2.0.<n>` , `latest` ; pushes to Docker Hub as `<svc>-2.0.<n>`	empty Docker Hub vars / push error
deploy-dev → test-dev	ArgoCD set / sync / wait healthy, then `tests/remote/test_dev_smoke.py`	unhealthy app / failing test
deploy-uat → test-uat	same for UAT, then `test_uat.py` (browse, cache, add/edit/delete, duplicate SKU = 409)	same
zap	OWASP ZAP passive baseline against uat.priartw.com	ZAP cannot run (findings do not fail: `-I`)
deploy-prod	ArgoCD set / sync / wait healthy	unhealthy app

> **No manual gate**
> As written, `deploy-prod` runs automatically once ZAP finishes. Add `when: manual` to the job if you want a human approval before production.

Environments are serialised with `resource_group` , so two pipelines never deploy to the same environment at once.

9 Verify each environment

Copy

```
kubectl -n dev get pods,svc,ingress
kubectl -n dev rollout status deploy/api deploy/frontend deploy/redis
kubectl -n dev rollout status statefulset/mysql

curl -s http://dev.priartw.com/api/health      # 200 when MySQL and Redis are up
curl -s http://dev.priartw.com/api/products    # "source":"db" then "cache"
argocd app get hpe-dev --plaintext --grpc-web
```

Repeat for `uat` and `prod`. Then open `http://prod.priartw.com/` in a browser: the catalog list, add, edit and delete should work.

Common problems

Symptom	Likely cause and fix
mysql pod `Pending`	PVC without a `gp3` StorageClass or EBS CSI driver (Step 2.2)
api `CreateContainerConfigError`	`app-secrets` missing in that namespace (Step 3)
api not Ready, `/health` 503	MySQL still initialising (readiness waits 20s+) or Redis password mismatch
`ImagePullBackOff`	Tag not on Docker Hub yet, or repo is private without a pull secret
publish fails: variables empty	Protected variable on an unprotected branch, or wrong name
deploy job: connection / gRPC errors	`ARGOCD_SERVER` wrong, ingress not reachable, or `server.insecure` not applied (restart argocd-server)
Pods `Pending` (too many pods)	Node pod limit (17 on t3.medium); add nodes or larger instances
Ingress 404 / default backend	DNS points elsewhere, or host in the overlay differs from the URL

Rollback

Copy

```
# redeploy an earlier published version
argocd app set hpe-prod --plaintext --grpc-web \
  --kustomize-image hpe-api=kharatramesh/hpe:api-2.0.<old> \
  --kustomize-image hpe-mysql=kharatramesh/hpe:mysql-2.0.<old> \
  --kustomize-image hpe-redis=kharatramesh/hpe:redis-2.0.<old> \
  --kustomize-image hpe-frontend=kharatramesh/hpe:frontend-2.0.<old>
argocd app sync hpe-prod --plaintext --grpc-web
```

10 Install monitoring and logging

Separate from the app and pipeline. Installs kube-prometheus-stack 92.3.1 (Prometheus, Alertmanager, Grafana, node-exporter, kube-state-metrics) and EFK (Elasticsearch, Kibana, Fluent Bit 0.58.3).

Copy

```
bash monitoring/install.sh
```

The script is re-runnable: it creates namespaces `monitoring` and `logging`, generates the Grafana and Kibana passwords (kept only in the cluster), runs `helm upgrade --install`, and applies the alert rules and the "HPE App" dashboard.

Logins

```
kubectl -n monitoring get secret grafana-admin -o jsonpath='{.data.admin-password}' | base64 -d
kubectl -n logging get secret kibana-basic-auth -o jsonpath='{.data.password}' | base64 -d
```

Copy

- **Grafana** `http://grafana.priartw.com` → Dashboards → *HPE App – dev / uat / prod*
- **Kibana** `http://kibana.priartw.com` → create a data view `k8s-*` with time field `@timestamp`; filter `kubernetes.namespace_name : "prod"`
- **Prometheus UI** is not exposed: `kubectl -n monitoring port-forward svc/kps-prometheus 9090`

Component	Storage	Retention
Prometheus	10Gi gp3	7 days
Grafana	5Gi gp3	persistent
Elasticsearch (single node, 1g heap)	20Gi gp3	CronJob deletes indices older than 7 days

> **Limits**
> The API has no `/metrics` endpoint, so request rate and latency are not charted. You get pod health, restarts, CPU, memory, network/disk I/O and request logs.

11 Using your own domain or registry

Values to change if you do not own `priartw.com` or the `kharatramesh` accounts:

What	File(s)
Hostnames dev / uat / prod	`k8s/overlays/*/kustomization.yaml`, `.gitlab-ci.yml` (environment urls, smoke-test `BASE_URL`, ZAP target)
ArgoCD, Grafana, Kibana hosts	`argocd/argocd-ingress.yaml`, `monitoring/prometheus/values.yaml`, `monitoring/efk/kibana.yaml`

Docker Hub repo	`DOCKERHUB_REPO` in `.gitlab-ci.yml`, `images` in `k8s/base/kustomization.yaml`, `docker-compose.yml`
Git repository URL	`repoURL` in `argocd/app-*.yaml`
StorageClass	`gp3` in `k8s/base/mysql.yaml` and the monitoring manifests (non-AWS clusters)

✓ Final checklist

- ☐ kubectl connected; ingress-nginx and gp3 StorageClass present
- ☐ Namespaces dev, uat, prod exist; `app-secrets` created in each
- ☐ ArgoCD running at argocd.priartw.com; three Applications registered
- ☐ GitLab variables set: DOCKERHUB_USER, DOCKERHUB_TOKEN, ARGOCD_SERVER, ARGOCD_AUTH_TOKEN
- ☐ DNS CNAMEs created for all six hostnames
- ☐ Pipeline green on master through deploy-prod
- ☐ `/api/health` returns 200 on dev, uat and prod
- ☐ Grafana dashboard shows data; Kibana data view `k8s-*` created
- ☐ Traffic is plain HTTP: add TLS before using real data